Desenvolvimento de Data-Driven Apps com Python [26E4_3]

Exibir Audima

TESTE DE PERFORMANCE - TP1 [OBRIGATÓRIO]

Olá, Bruno.

Chegamos em uma das etapas de preparação! A cada Teste de Performance (TP) você terá a oportunidade de praticar os conhecimentos adquiridos e receber feedbacks relevantes para o seu aprendizado.

Uso de IAs: Sinal Vermelho 🔴

Todas as partes deste trabalho devem ser da autoria do aluno. Qualquer uso de ferramentas generativas de IA, como ChatGPT, Gemini ou Claude, é estritamente proibido. O uso de IA generativa será considerado má conduta acadêmica e estará sujeito à aplicação do código disciplinar, pois as tarefas deste trabalho foram elaboradas para desafiar o aluno a desenvolver conhecimentos de base, pensamento crítico e habilidades de resolução de problemas. O uso da tecnologia de IA limitaria sua capacidade de desenvolver essas competências e de atingir os objetivos de aprendizagem desta disciplina.

Exercício 1 Configurar ambiente de desenvolvimento para uso de FastAPI

Instale o **FastAPI** e o servidor **Uvicorn** no seu ambiente local. Crie um ambiente virtual e instale as bibliotecas necessárias para a implementar uma biblioteca FastAPI. Crie um arquivo requirements.txt contendo todas as bibliotecas instaladas.

Exercício 2 Construir uma aplicação simples usando FastAPI

Crie uma aplicação básica em FastAPI que exiba a mensagem "Hello, FastAPI!" na rota principal (/).

Exercício 3 Criar rotas em uma aplicação FastAPI

Adicione mais duas rotas na sua aplicação:

- Uma rota /status que retorna uma mensagem indicando que o servidor está funcionando.
- Uma rota /user/{username} que receba um parâmetro na URL e retorne uma saudação personalizada para o usuário.

Exercício 4 Organizar projetos com FastAPI

Estruture seu projeto FastAPI de forma organizada, separando o código em múltiplos arquivos:

- Um arquivo para as rotas (routes.py).
- Um arquivo principal para a execução do app (main.py).

Exercício 5 Manipular as respostas usando Response Models

Crie um **Response Model** para a rota /user/{username} que retorne um JSON com o nome do usuário e uma mensagem. Exemplo de resposta:

```
{
  "username": "john",
  "message": "Welcome, john!"
}
```

Exibir Audima

Exercício 6 Tratar erros através de exceções HTTP

Implemente o tratamento de erro para a rota /user/{username}, retornando uma mensagem de erro 404 se o nome de usuário não for encontrado. Utilize o modelo de exceção HTTP: **from fastapi import HTTPException**.

Exercício 7 Criar rotas POST em FastAPI

Adicione uma rota **POST** chamada /create-user que receba um nome de usuário e idade via JSON no corpo da requisição, e retorne os dados recebidos. Exemplo de corpo da requisição:

```
{
  "username": "john",
  "age": 30
}
```

Exercício 8 Validar dados de entrada com Pydantic

Utilize o **Pydantic** para validar a entrada de dados na rota /create-user. O nome de usuário deve ser uma string e a idade um número inteiro positivo. Caso os dados sejam inválidos, a API deve retornar um erro apropriado. Todos os modelos devem estar no arquivo models.py.

Exercício 9 Criar múltiplos métodos HTTP para uma rota

Crie uma rota /item/{item_id} que suporte os métodos **GET** e **DELETE**:

- **GET**: Retorna o item correspondente ao item_id.
- **DELETE**: Remove o item do sistema e retorna uma mensagem de confirmação.

Exercício 10 Tratar erros de validação com exceções personalizadas

Implemente uma exceção personalizada para tratar erros de validação na rota /item/{item_id}, retornando uma mensagem de erro 400 caso o item_id seja inválido.

Exercício 11 Explicar a importância de APIs no contexto da ciência de dados

Crie uma rota **POST** chamada /birthday que receba um **nome** e uma **data de aniversário** (no formato "YYYY-MM-DD") via JS corpo da requisição. A rota deve calcular quantos dias faltam até o próximo aniversário da pessoa e retornar uma mensagem personalizada.

Exercício 12 Criar documentação automática com FastAPI

Use o **Swagger UI** integrado ao FastAPI para gerar automaticamente a documentação das rotas criadas neste exercício.

Regras de Entrega

Organização dos arquivos

Exibir Audima

A atividade deverá ser entregue em um único arquivo compactado (.zip) nomeado conforme a regra "**nome_sobrenome_DR3_TP1.zip**", adicionando o arquivo '**.py**' ou '**.ipynb**', contendo o projeto desenvolvido em Python com FastAPI. O projeto deverá apresentar, no mínimo, a seguinte estrutura:

```
fastapi_exercicios/
├── main.py
├── routes.py
├── models.py
└── requirements.txt
```

Descrição dos arquivos:

- main.py: responsável pela inicialização e configuração da aplicação FastAPI.
- routes.py: contém as rotas e suas respectivas implementações.
- models.py: contém os modelos Pydantic utilizados na validação de dados e na definição das respostas.
- requirements.txt: apresenta as dependências necessárias para executar a aplicação.

Implementação dos exercícios

- Todos os 12 exercícios deverão ser implementados em uma única aplicação FastAPI.
- As rotas deverão seguir os nomes, métodos HTTP e comportamentos definidos nos enunciados.
- O código deverá estar organizado em funções, evitando repetições desnecessárias.
- Os modelos Pydantic devem ser utilizados nos exercícios que exigem validação de dados e definição de respostas.
- Os tratamentos de erros deverão retornar os códigos HTTP apropriados.
- O código deverá ser executável e não poderá conter erros de sintaxe.
- **Não incluir:** pastas de ambientes virtuais (venv, .venv), arquivos temporários, diretórios __pycache__ ou outras dependências instaladas localmente.
- **Os trabalhos que não seguirem as regras de entrega não serão avaliados.**

Bons estudos!

Status da entrega

Status da entrega	Nenhuma tentativa
Status da avaliação	Não avaliado
Data de entrega	segunda, 26 out 2026, 23:59
Tempo restante	8 horas 6 minutos

Rubrica

Template de Rubrica para ser utilizado com a extensão Rubricator

1. Desenvolver APIs utilizando o framework FastAPI **O aluno configurou o ambiente de desenvolvimento com FastAPI e Uvicorn e apresentou evidências da instalação bem-sucedida?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno criou uma aplicação que exibe a mensagem "Hello, FastAPI! " corretamente e apresentou a execução no navegador como evidência?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno criou uma rota "/status " que retorna o status do servidor?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno criou uma rota "/user/{username} " que retorna uma saudação personalizada?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno organizou o projeto em múltiplos arquivos, incluindo rotas e arquivo principal?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno explicou de forma clara e objetiva a importância da organização do projeto?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno utilizou Response Models corretamente para retornar um JSON com o nome do usuário e uma mensagem?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno implementou o tratamento de erros com exceções HTTP adequadamente?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno implementou uma rota POST chamada "/create-user " que recebe nome de usuário e idade via JSON?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno utilizou Pydantic corretamente para validar os dados de entrada na rota "/create-user "?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno criou uma rota "/item/{item_id} " que suporta os métodos GET e DELETE corretamente?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno tratou corretamente os erros de validação na rota "/item/{item_id} " com exceções personalizadas?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica
1. Desenvolver APIs utilizando o framework FastAPI **O aluno criou e implementou uma rota "/birthday " que calcula corretamente os dias até o próximo aniversário, utilizando um código Python correto e funcional?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica

Exibir Audima

1. Desenvolver APIs utilizando o framework FastAPI **O aluno configurou a documentação automática no Swagger UI e apresentou a interface gerada para demonstrar a correta configuração e acesso às rotas?**	Não demonstrou o item de rubrica	Demonstrou o item de rubrica

Última modificação	-
Comentários da entrega	Comentários (0)

Exibir Audima

ADICIONAR ENTREGA DO TRABALHO

Você ainda não fez um envio.

PÁGINA ANTERIOR

Seguir para...

PRÓXIMO PÁGINA